

Functions

Jarmo Hurri

Introduction

- programmers strive to avoid writing the same code twice
A good programmer is lazy.
- there are many reasons for this
 - saving time: soon you are writing it the third time
 - (often) shorter code
 - easier to understand: "same thing happens here"
 - easier to maintain: only change one location
 - easier to debug: isolates problem to one place
 - path to abstraction: patterns emerge
- if you are repeating yourself, think again and generalize
- functions are a fundamental method of generalization
- long code written only once is often split into procedures
 - structuring
 - division of labour

Function (or method or procedure)

- reusable piece of code
- defined interface to caller
- operation is given a name, making it easier to understand
- hides details and complexity inside
- hides variables etc. inside
- supported by hardware (more on this later)
- these are called
 - *functions* in most programming languages
 - *methods* in object-oriented programming
 - *(sub-)procedures* in general programming terminology
- *procedural programming*: related programming paradigm

Example 1

- define a function with *signature*
`abs_value(x)`
that returns the absolute value of its numeric parameter `x`
- use this function to define another function
`distance(a, b)`
that returns the distance between parameters `a` and `b` on the number line
- (note: `abs` is available as a builtin function)

Features of Python functions

- definition starts with keyword `def`
- *body* of function is in a block (suite)
- value returned using keyword `return`

- ends execution of function

- *signature* of function identifies

- name of function

- parameters

- names
 - number
 - order

- additional properties (positional, keyword, default value)

- example

distance (a, b)
name parameters

Example 2

Write a function

```
interleave(a, b)
```

that returns a list whose elements are a and b interleaved:

```
[a[0], b[0], a[1], b[1], ...]
```

You may assume that a and b have the same number of elements.

Argument passing in Python

- in a function call, values of parameters are called *arguments*
- in Python, argument passing is by *object reference*
 - parameter is bound to address where object lives
- inside function
 - arguments with *mutating* methods e.g. `append()` can be modified
 - arguments of *immutable* types (no mutating methods) e.g. `int` can *not* be modified
- example ↪

```
def f(lst, x):  
    x *= 5 # just rebinds x, does not mutate  
    lst.append(x) # mutates  
lst = list(range(5))  
x = 11  
f(lst, x)  
print(lst, x)
```

Example 3

Write a function

```
swap(lst, ind1, ind2)
```

which swaps the contents of `lst[ind1]` and `lst[ind2]`.

Example 4

Write a function with signature

```
read_floats(n)
```

which takes a nonnegative integer parameter `n` and reads `n` floats from the user. The function returns these floats in a list. Each read float is prompted of the following form; here `n=6`:

input value 1 out of 6:

Scope

- variable names, function names etc. are *identifiers*
- *scope* of identifier is region of code where identifier is visible
- scope of identifier is *within its block after bound*
- in Python, following constructs create new scopes
 - Python file (module)
 - function (`def`)
 - class (later)
- so identifier can be visible within file, function or class
 - note: e.g. `if` or `while` do *not* create scopes

Example 5

```
g = 100 # global scope: visible everywhere below

def f(x): # local scope: visible in function
    y = 20 # local scope
    return g + x + y

print(f(3))
try:
    print(x, y) # error: x, y not in scope
except NameError:
    print('got name error')
```

123

got name error

Namespace resolution

- what if, say, `x` is bound in multiple nested scopes
- *namespaces* contain bindings of identifiers to objects
 - each scope has its own namespace
- namespace resolution rules specify order of search
- in Python, namespace search order is
 - L local block (including parameters)
 - E enclosing (outer) blocks
 - G global namespace (not inside any block)
 - B builtin namespace
- notes (*)
 - search can be overruled with keywords `global` and `nonlocal`
 - rules are different for value assignment vs. access

Example 6

```
# global variables (bindings in global namespace)
g = -1
y = -2
x = -3
def f(x):
    def h(y):
        # now y bound locally, x in enclosing, g globally
        # and print() is builtin
        print(f'g={g}, y={y}, x={x}')
    return h # note: f returns a function

func = f(1)
func(2)
```

g=-1, y=2, x=1

Homework 1

1. Explain three benefits of using functions.
2. What is the signature of a function?
3. Consider the following function:

```
foo(a, b)
```

- 3.1 What is the name of the function?
- 3.2 What is the number of parameters?

Homework 2

1. Write a function with signature

`sum_upto(n)`

which returns the value $\frac{n(n+1)}{2}$. (Note: this is the value of $1 + 2 + \dots + n$.)

2. Use the function in a program that prints the values of this function for $n = 1, 2, \dots, 20$.

Homework 3

Write function

`read_ints()`

which reads integers from the user. The function returns the integers in a list. End of input is indicated by an empty input (just pressing return/enter). You can prompt the user with any sensible prompt string.

Note: The function `input()` returns an empty string if the user provides an empty input.

Homework 4

1. Write a function with signature

`share(a, b)`

which returns True if any of the elements (at least one) of list a is equal to any of the elements (at least one) of list b. Otherwise the function returns False.

2. Test your function `share()` in a program.

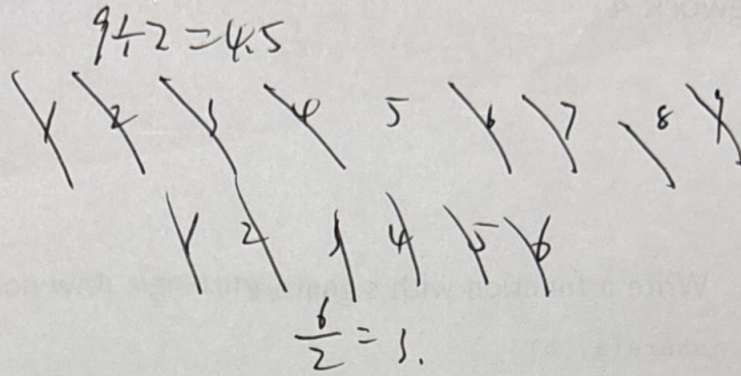
Homework 5

Consider the following procedure REMOVE.

- REMOVE takes three parameters
 - an integer list ORIG of length 10 with positive integer values
 - a positive integer X
 - an integer list OUT of length 10.
- REMOVE replaces OUT with version of ORIG with elements equal to X removed and zeros added to the end. For example, with
 - ORIG=[1, 2, 3, 4, 5, 5, 4, 3, 2, 1]
 - X=4
 - OUT=[6, 6, 6, 6, 6, 6, 6, 6, 6, 6]at the return of the procedure, the value of OUT would be
 - OUT=[1, 2, 3, 5, 5, 3, 2, 1, 0, 0]

Write a corresponding function and use it in a program.

Homework 6



Write function

`alternate(lst)`

which returns a new list with the elements of `lst` alternating as follows: first, last, 2nd, "2nd last" etc.

Homework 7

In Python, functions are *first-class citizens*: they are treated like any other data type. Identifiers can be bound to functions, functions can be passed as parameters, functions can be elements of lists etc.

1. Write a function

`iterate(f, x, n)`

returning the list

$$[f(x), f(f(x)), \dots, \underbrace{f(f(f(\dots(f(x))\dots)))}_{n \text{ times}}]$$

Call the function with $f(x) = 0.5(x + \frac{2}{x})$, $x = 1$ and $n = 6$.

Homework continues

2. Write a function

```
apply_functions(fs, x)
```

where

$$fs = [f_1, f_2, \dots, f_n]$$

returning the value

$$f_1(f_2(\dots(f_n(x)\dots))).$$

Call the function with

```
fs = ['...'.join, str.split, str.lower]
```

and

```
x = 'WHAT IS THIS?'
```